



# NightFlow

ISPW PROJECT -University of Rome Tor Vergata.

MARTINA MAURIZI

### INDEX

---

|                                                   |           |
|---------------------------------------------------|-----------|
| NightFlow.....                                    | 1         |
| NightFlow-PROJECT DOCUMENTATION.....              | 2         |
| <b>1. Software Requirement Specification.....</b> | <b>3</b>  |
| 1.1 Introduction .....                            | 3         |
| 1.1.1 Aim of the Document.....                    | 3         |
| 1.1.2 Overview of the Defined System .....        | 3         |
| 1.1.3 Hardware e Software Requirements.....       | 3         |
| 1.1.4 Related Systems – Pros and Cons .....       | 4         |
| 1.2 User Stories.....                             | 5         |
| 1.3 Functional Requirements .....                 | 5         |
| 1.4 Use Cases .....                               | 6         |
| 1.4.1 Overview Diagram.....                       | 6         |
| 1.4.2 Internal steps: .....                       | 7         |
| <b>2. Storyboards.....</b>                        | <b>8</b>  |
| <b>3. Design.....</b>                             | <b>11</b> |
| 3.1 Class Diagram.....                            | 11        |
| 3.1.1 VOPC analysis .....                         | 11        |
| 3.1.2 Design-Level Diagram.....                   | 12        |
| 3.2 Design Patterns .....                         | 14        |
| 3.3 Activity diagram .....                        | 15        |
| 3.4 Sequence diagram .....                        | 16        |
| 3.5.State diagram .....                           | 17        |
| <b>4. Testing.....</b>                            | <b>18</b> |
| <b>5. Code.....</b>                               | <b>19</b> |
| <b>6. Video.....</b>                              | <b>19</b> |

# **1. Software Requirement Specification**

## **1.1 Introduction**

### **1.1.1 Aim of the Document**

The primary purpose of this document is to present the fundamental and specific requirements of the NightFlow system. It defines the features offered to the various system participants, the functional requirements, the operational flows, and the logical architecture adopted. All the key internal and external aspects of the system will then be examined, with the aid of several diagrams.

### **1.1.2 Overview of the Defined System**

NightFlow is an application designed to manage ticket reservations for private events. The system offers flexibility in managing payments and access types, ensuring a seamless experience for both organizers and end users.

Key features include:

- **Event Management:** Organizers can create and edit private events.
- **Booking System:** Clients can reserve tickets for selected events and view their details.
- **Client List Management:** Tools dedicated to organizers for viewing, monitoring, and managing attendees registered for their events.
- **Ticket Management:** Different ticket types can be provided (e.g., standard access, reserved areas).
- **Payment Management:** Support for ticket payments, both "on-site" (at the event) and via secure online payments.
- **Email notifications** for booking confirmations and cancellations.

There are two user types with different functionalities: Clients and Organizers.

### **1.1.3 Hardware e Software Requirements**

#### **Software requirements:**

- Java 17+
- Maven 3.8+
- MySQL Server 8.0+
- JavaFX 17+
- IDE: IntelliJ IDEA (recommended) or Eclipse
- Draw.io (for creating UML diagrams)

**Hardware requirements:**

- CPU: Dual-Core 2.0 GHz
- RAM: 4 GB (2 GB minimum)
- Disk Space: 1 GB
- Operating System: Windows, MacOS, Linux (compatible with java 17+)

**1.1.4 Related Systems – Pros and Cons****NightFlow:**

- **Pros:** Completely free system, with no transaction fees, accessible to all organizers and users without financial barriers and flexible payment options for bookings
- **Cons:** Lack of social/marketing integration, Desktop only, no mobile application available and Push notifications on smartphones.

**- Whova:**

- **Pros:** It offers additional features such as designing and printing name badges and promoting interaction and networking among participants.
- **Cons:** It may not be the best choice for small events or those with basic needs.

**- Eventbrite:**

- **Pros:** Eventbrite's user interface is intuitive and easy to navigate, making it simple to create and manage events. It also has a large base of users and a strong online presence, which means your events can reach a wide audience.
- **Cons:** It charges fees for ticket sales, which can be significant for large events, and you don't have full control over the branding of event pages because Eventbrite also promotes its brand.

## 1.2 User Stories

1. As an attendee, I want to purchase an event ticket directly from the system, so that I can avoid waiting in line at the entrance.
2. As an attendee, I want to receive a confirmation after booking, so that I can be sure my ticket has been reserved correctly.
3. As an organizer, I want to view the history of my past events, so that I can remember which evenings I've organized.

## 1.3 Functional Requirements

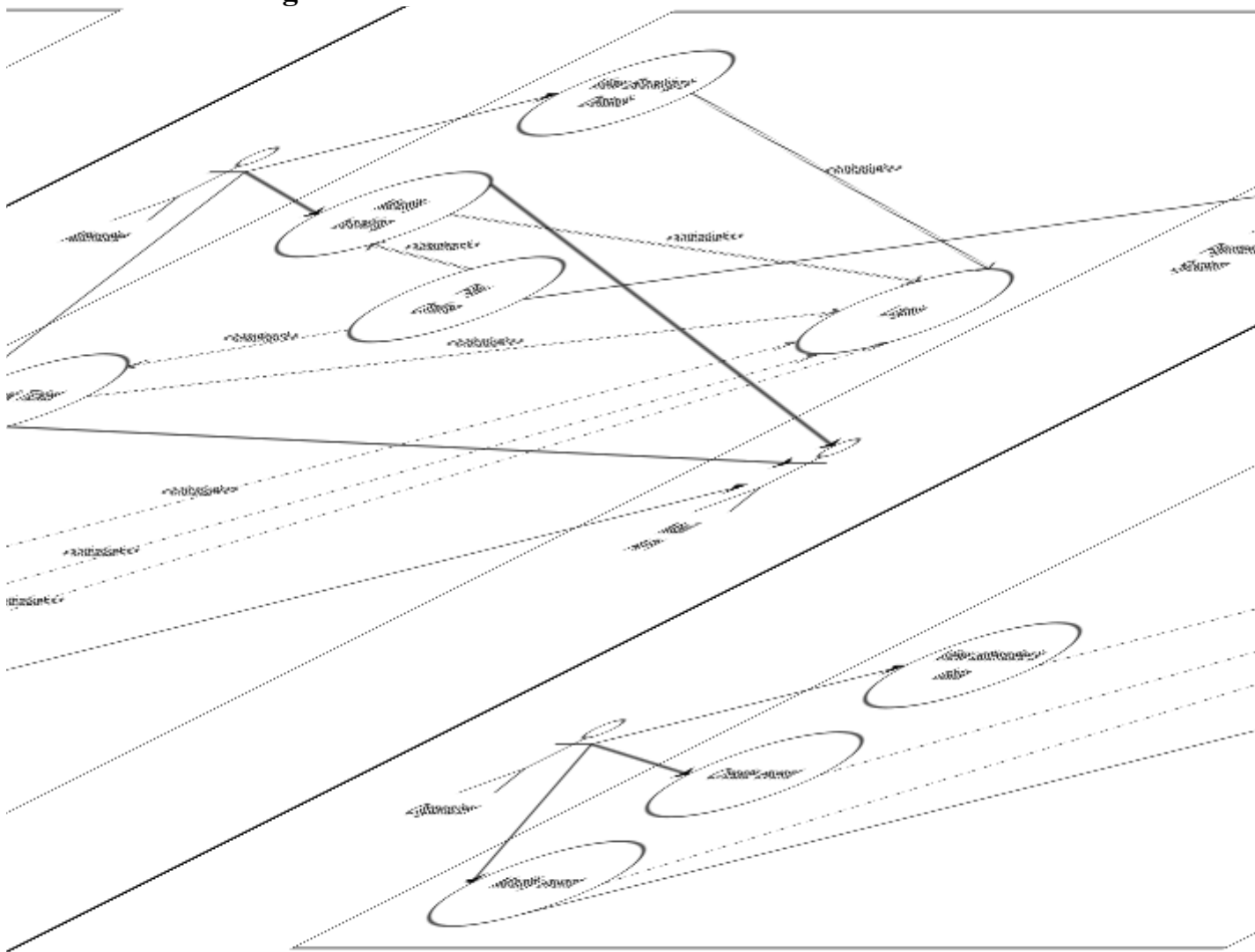
1. The system shall support an online payment\* system for ticket bookings.
2. The system shall provide a notification system\* to send a confirmation message to the attendee upon completion of the reservation.
3. The system shall provide the organizer the opportunity to consult the history of their past events.

### Glossary :

- **payment system\***: External gateway (e.g., PayPal) for processing and validating credit card transactions.
- **notification system\***: Service for sending emails(e.g., SendGrid) containing details and ticket code.

## 1.4 Use Cases

### 1.4.1 Overview Diagram



### 1.4.2 Internal steps:

#### Use case: **Book Ticket (Attendee)**

1. The attendee accesses their account by UC Login.
2. The system displays a list of available events.
3. The attendee selects “Book Ticket” for the desired event.
4. The attendee selects the ticket type and selects "Check Out”.
5. The attendee selects the payment method and enters the data required for the booking.
6. The system displays the booking summary.
7. The attendee confirms the booking.
8. The system sends a confirmation mail both to the attendee and the organizer.
9. The system adds the booking to the attendee's confirmed bookings and the use case ends.

#### -Extensions

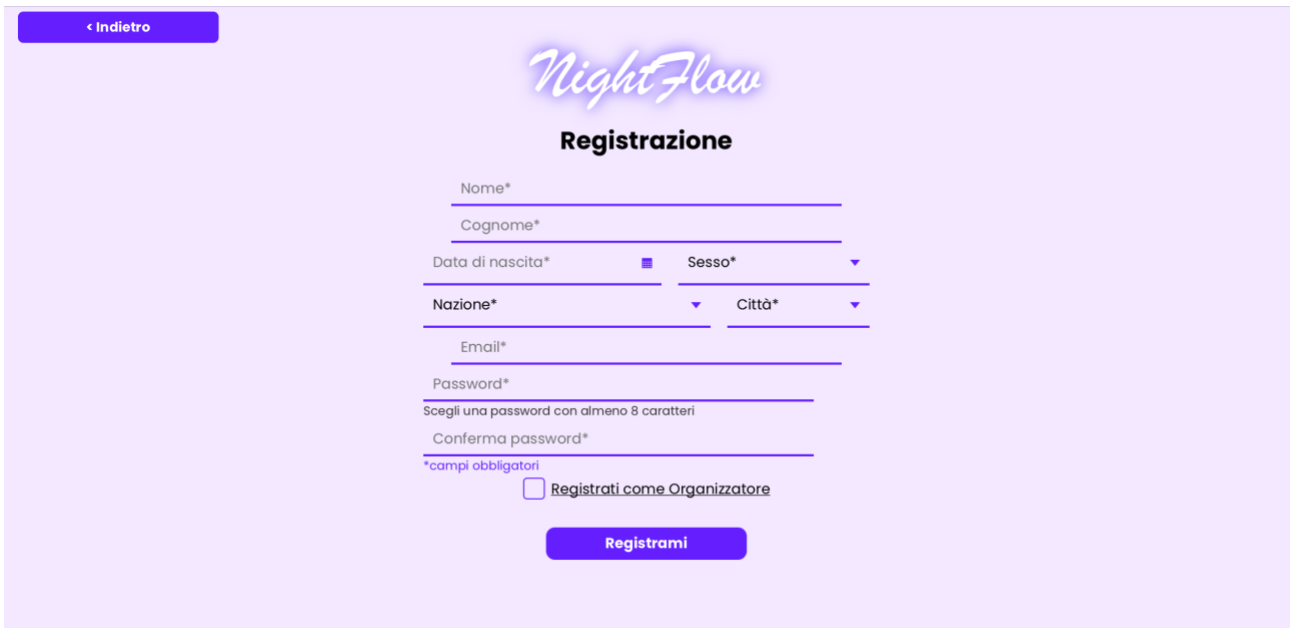
4a. *Event sold out*: The system displays the message "Event SOLD OUT" and returns the attendee to the ticket selection screen (step 3).

5a. *Attendee already booked*: The system displays the message "Already a booking for this event" The use case ends.

5b. *Online payment*: If the customer selects an online payment method instead of an on-site payment, the flow pauses temporarily, and the system executes the extended use case "Online payment". After successful payment, the flow resumes from step 6.

## 2. Storyboards

### REGISTRAZIONE



The registration form for NightFlow is displayed on a light purple background. At the top left, there is a blue button labeled '< Indietro'. The NightFlow logo is centered at the top. Below the logo, the title 'Registrazione' is centered. The form consists of several input fields: 'Nome\*' (text), 'Cognome\*' (text), 'Data di nascita\*' (date picker), 'Sesso\*' (dropdown menu), 'Nazione\*' (dropdown menu), 'Città\*' (dropdown menu), 'Email\*' (text), 'Password\*' (text), and 'Conferma password\*' (text). A note below the password fields states 'Scegli una password con almeno 8 caratteri'. A checkbox labeled '\*campi obbligatori' is present, followed by a checkbox labeled 'Registrati come Organizzatore'. At the bottom, there is a blue button labeled 'Registrarmi'.

< Indietro

*NightFlow*

**Registrazione**

Nome\*

Cognome\*

Data di nascita\* Sesso\*

Nazione\* Città\*

Email\*

Password\*

Scegli una password con almeno 8 caratteri

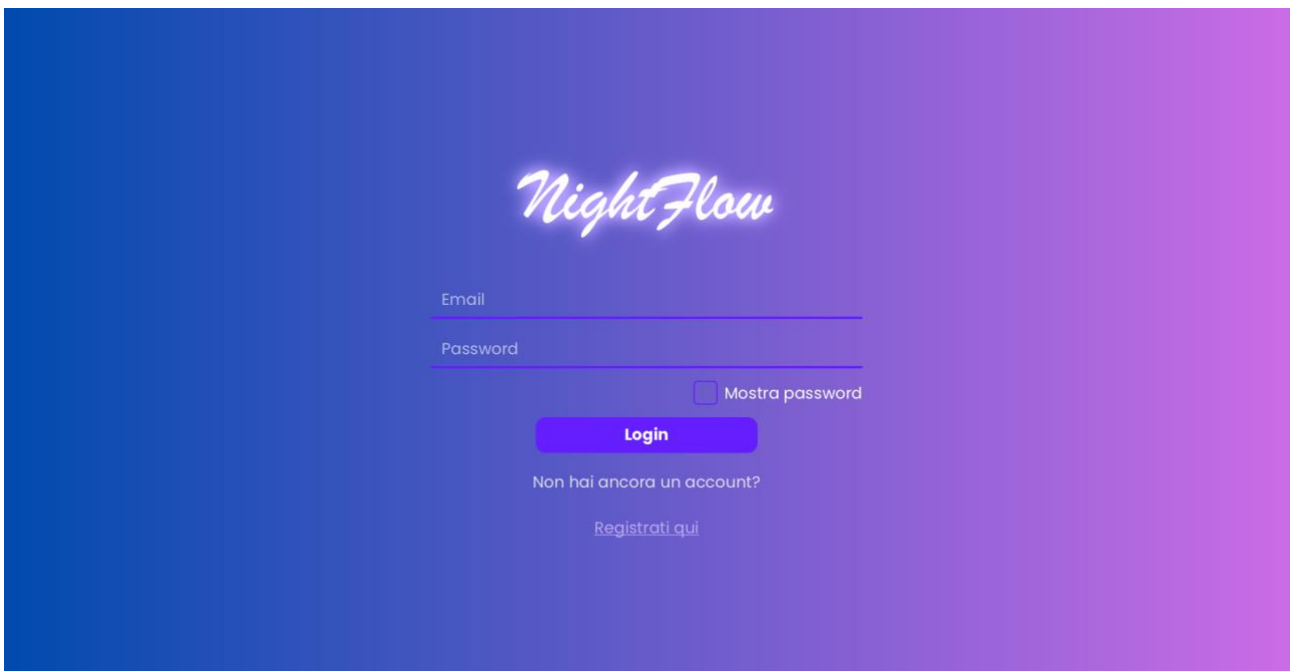
Conferma password\*

\*campi obbligatori

☐ Registrati come Organizzatore

Registrarmi

### LOGIN



The login form for NightFlow is displayed on a blue and purple gradient background. The NightFlow logo is centered at the top. Below the logo, there are two input fields: 'Email' and 'Password'. A checkbox labeled 'Mostra password' is located to the right of the password field. Below the input fields, there is a blue button labeled 'Login'. Underneath the button, the text 'Non hai ancora un account?' is displayed, followed by a link labeled 'Registrati qui'.

*NightFlow*

Email

Password

☐ Mostra password

Login

Non hai ancora un account?

[Registrati qui](#)



## DASHBOARD ORGANIZER

NightFlow - Area Organizer



Log out

### Benvenuto, Organizzatore!

Cosa vuoi fare oggi?

+ Crea Evento

= Modifica Eve...

Gestisci Lista

## DASHBOARD CLIENT

NightFlow



Cerca



Log out

### Prossimi eventi



11

LUG

**Jolie Club**

Via Velletri 13, Roma

Ingresso in lista

PRENOTA BIGLIETTO



12

LUG

**Amazonia**

Via del Santuario, Roma

Ingresso in lista

PRENOTA BIGLIETTO



11

**Magazzini Generali**



15

**The sanctuary eco retreat**

## BOOK TICKET(SELECT TYPE)

[< Indietro](#) *NightFlow*

  [Log out](#)

### Cena Buffet

 Jerò Restaurant - Via del Santuario, Roma



☐ Liste

#### Ingresso

|              |             |          |                       |
|--------------|-------------|----------|-----------------------|
| 22:00 - Late | Senza drink | 25,00 €  | <input type="radio"/> |
| 22:00 - Late | Con drink   | 30,00 €  | <input type="radio"/> |
| 22:00 - Late | Tavolo VIP  | 110,00 € | <input type="radio"/> |

Check Out

## CHECK OUT

[< Indietro](#) *NightFlow*

  [Log out](#)



**JERÒ RESTAURANT - Cena Buffet**  
**17 LUG**  
Riepilogo : Senza drink

#### I tuoi Dati

Anna

anna@demo

05/08/1998

Telefono\*

\*campi obbligatori

#### Scegli il metodo di pagamento

 **Carta di debito o credito**

Nome sulla carta

Numero della carta

Scadenza(MM/AA) CVV

PayPal

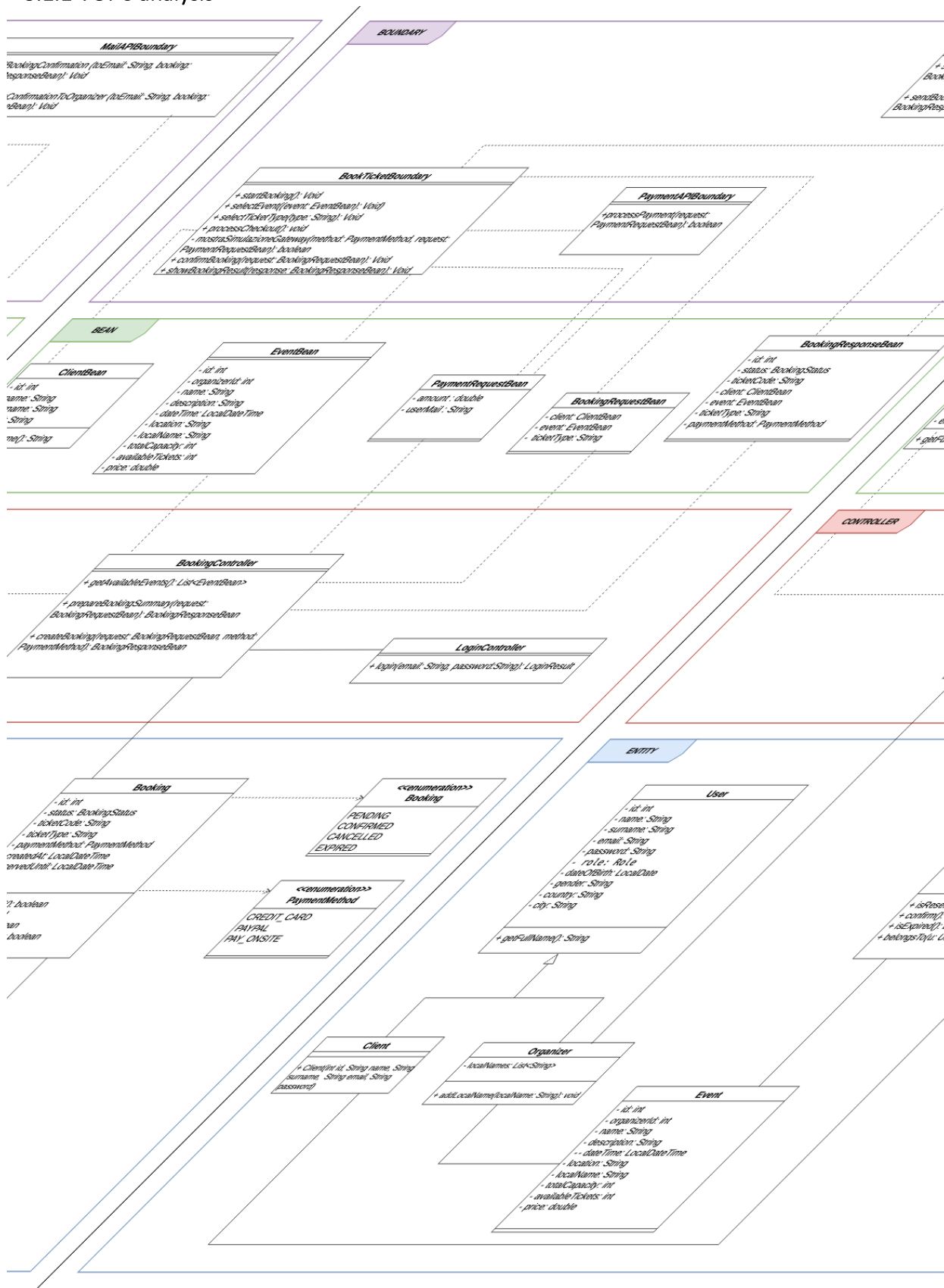
Paga all'ingresso

Conferma Prenotazione

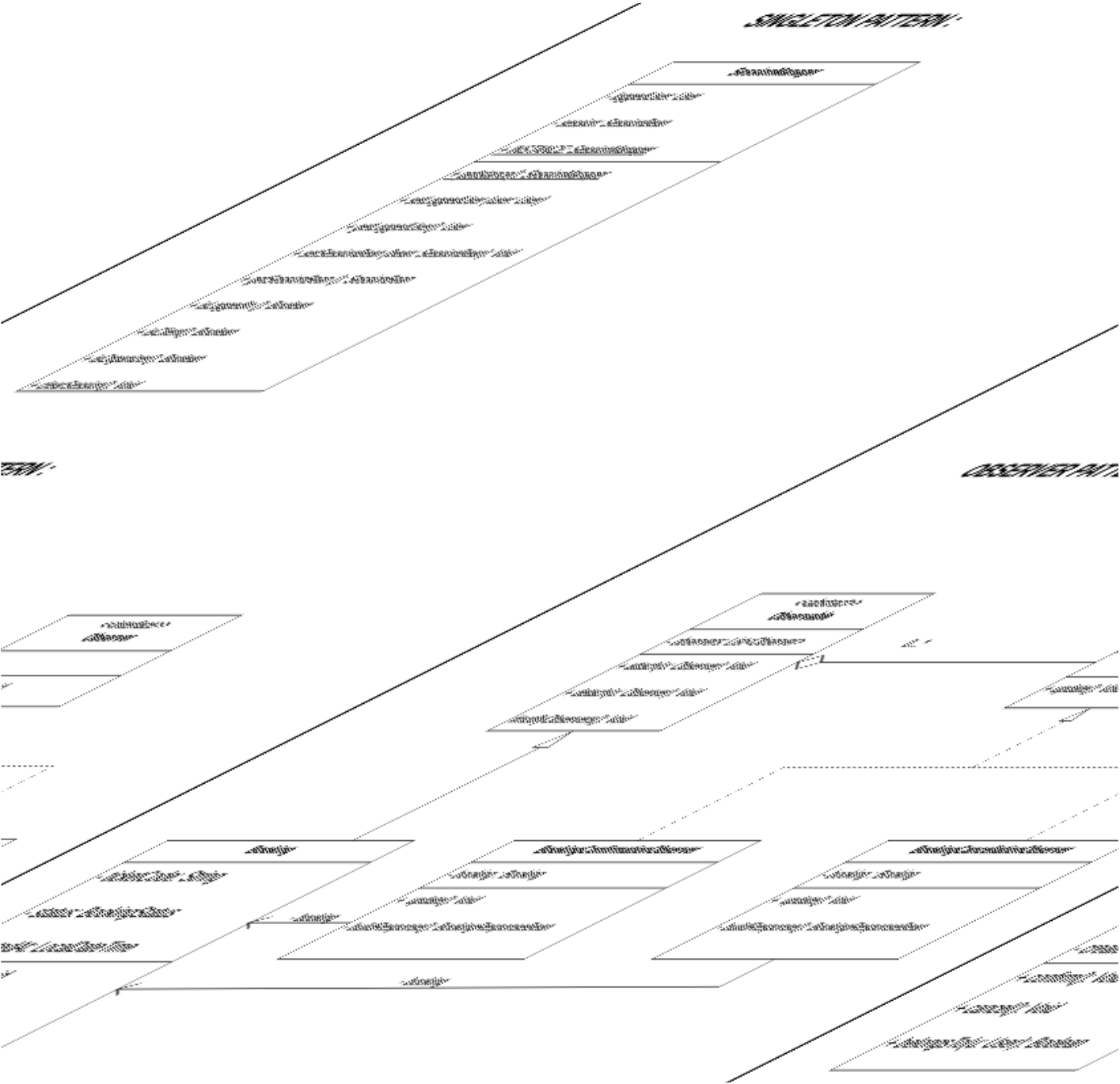
## 3. Design

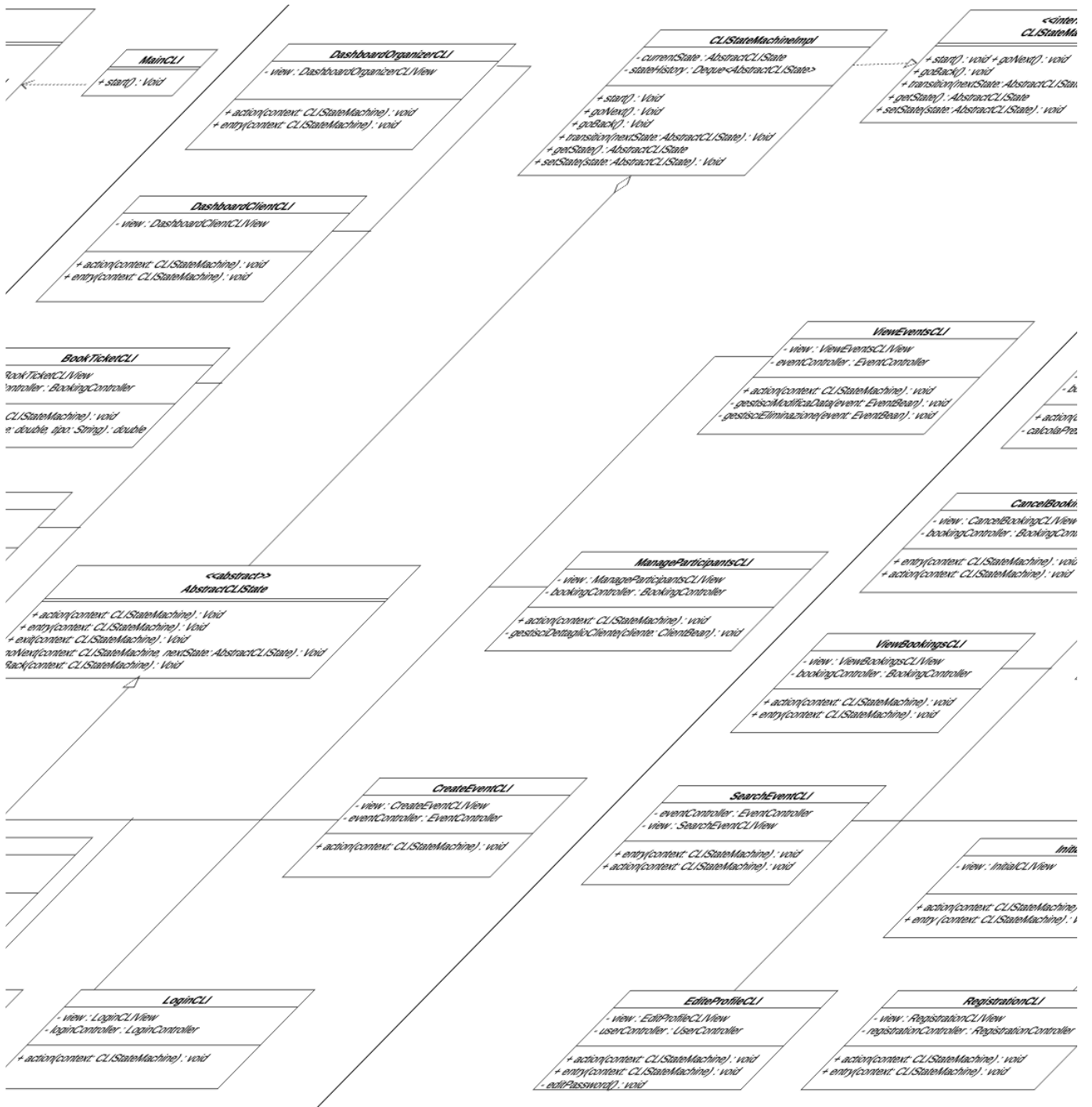
### 3.1 Class Diagram

#### 3.1.1 VOPC analysis



3.1.2 Design-Level Diagram





## 3.2 Design Patterns

### 1. SINGLETON

I implemented the **Singleton** pattern for `SessionManager` to establish a single, shared source of truth for session data (such as the authenticated user and their role). This approach eliminates the need to manually pass session objects across controllers, while ensuring that exactly one instance exists throughout the application's lifecycle.

The implementation follows the "**Initialization-on-demand holder**" idiom. By utilizing a private static inner class, I achieved lazy initialization and inherent thread safety without the performance overhead of explicit `synchronized` blocks. Furthermore, `SessionManager` centralizes authorization logic; by exposing dedicated role-checking methods, it allows controllers to verify permissions without requiring direct access to the `User` object.

### 2. OBSERVER

To manage notifications regarding booking state changes, I adopted the **Observer** pattern. This design effectively decouples the `Booking` entity and `BookingController` from the `NotificationService`.

In this setup, the `BookingController` attaches the appropriate observers before triggering `confirm()` or `cancel()` actions. Upon a state change, the concrete observers `BookingConfirmationObserver` and `BookingCancellationObserver` automatically react to notify both the attendee and the organizer. This architecture is highly extensible: should the need arise to incorporate new communication channels (e.g., SMS or push notifications), we can simply introduce new observers without modifying existing business logic.

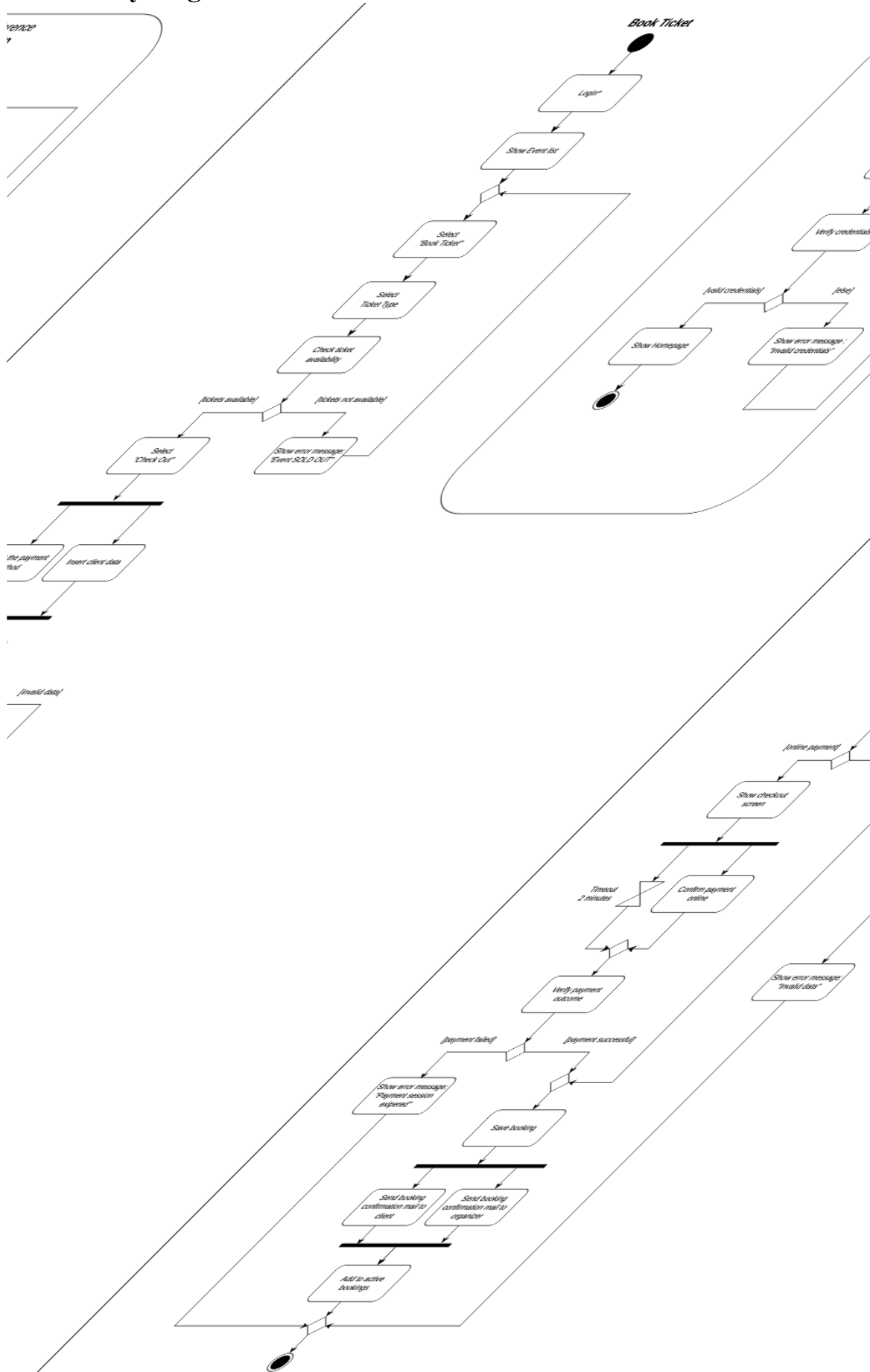
### 3. STATE

To replace the rigid, monolithic navigation logic previously handled by a complex `switch-case` block in the `MainCLI`, I implemented the **State** pattern.

By transitioning to an object-oriented approach, I encapsulated each screen into a dedicated class extending `AbstractCLIState`. Each state manages its own lifecycle through three primary methods: `entry()` for initialization, `action()` for handling inputs and transition logic, and `exit()` for cleanup.

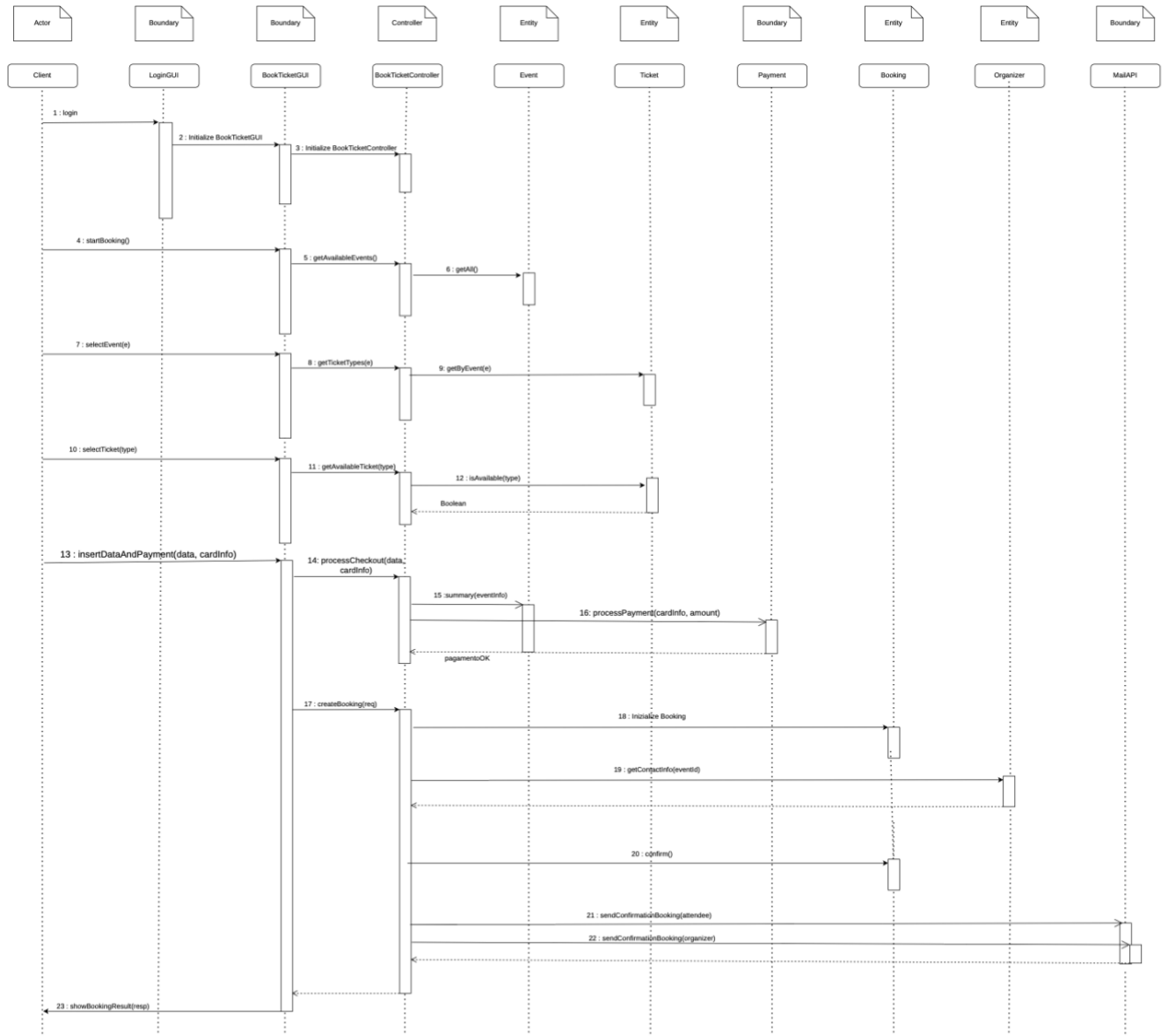
`CLIStateMachineImpl` acts as the context, maintaining the current state and a history stack (`Deque`) to enable seamless forward and backward navigation. Consequently, every screen is now a self-contained, autonomous module, making the system easy to extend while strictly adhering to the *Open-Closed* principle.

### 3.3 Activity diagram



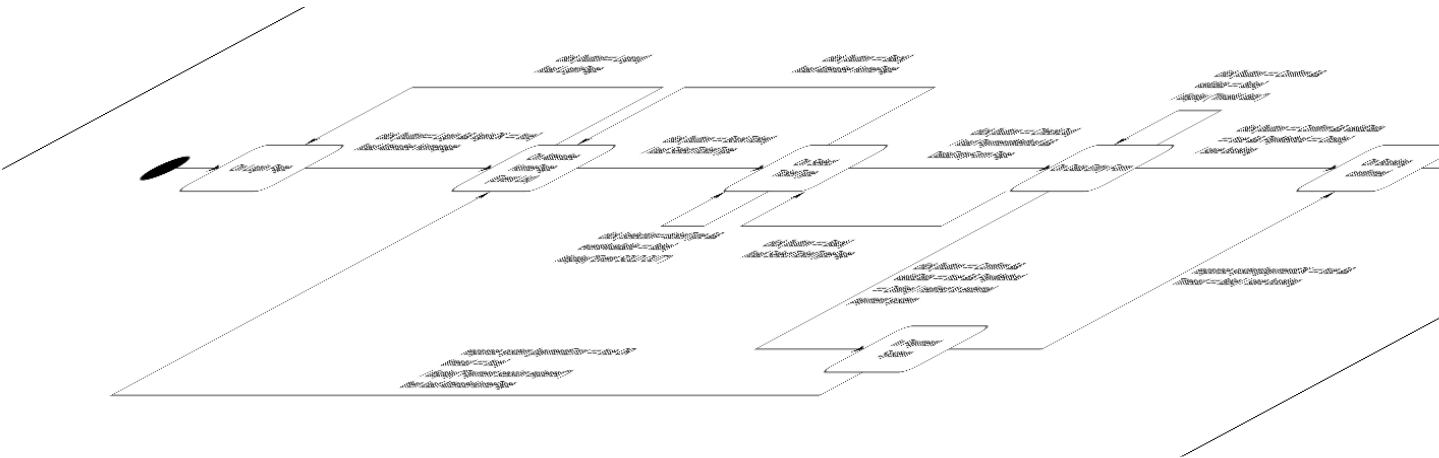
### 3.4 Sequence diagram

#### Use case : BookTicket





3.5 State diagram  
Use case : BookTicket



## **4. Testing**

### **1. Booking on an already reserved event**

The test verifies the "Soft Lock" mechanism for ticket bookings. When User 1 initiates the booking process (prepareBookingSummary), the system temporarily reserves the ticket. A second attempt by User 2 on the same event correctly raises a BookingException, preventing simultaneous access to limited resources until the lock is released or the session expires.

### **2. Login with invalid credentials**

The authentication system was tested by attempting to log in with an email address does not present in the data store (utilizing the MEMORY persistence mode to isolate business logic). The test confirms that the system correctly rejects unauthorized access by raising a LoginException, ensuring that only registered users can access the platform.

### **3.Rejection of duplicate registrations**

The system was verified to prevent two accounts from sharing the same email address. After a first successful registration, a second attempt with the same email must raise a RegistrationException, ensuring the uniqueness of user identities within the platform.

## **5.Code**

GitHub :

<https://github.com/MartinaMaurizi/NightFlow>

Sonar Qube Cloud:

[https://sonarcloud.io/project/overview?id=MartinaMaurizi\\_NightFlow](https://sonarcloud.io/project/overview?id=MartinaMaurizi_NightFlow)

## **6. Video**

<https://youtu.be/w-RNxzK7c6I?si=E9p9q9pou5HWgwCl>